Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Roll no:	CS3807 – Deep Learning Laboratory 24011101036	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and tra difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VG GNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3 × 3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3 × 3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1, 3×3, 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficu ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly th the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

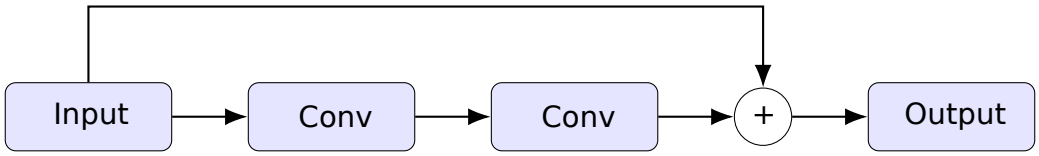

Figure 1:Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing number of parameters. The dilation rate D determines spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

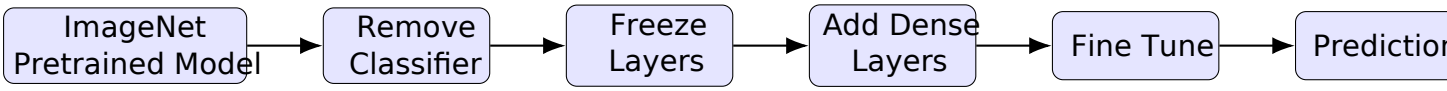

Figure 2:Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Number of Classes: 10
- Image Size: 32 × 32 × 3
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range [0,1].
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

17. Mandatory Plots

Include the following plots in the laboratory report.

1. Sample CIFAR-10 images
2. Training Accuracy
3. Validation Accuracy
4. Training Loss
5. Validation Loss
6. Confusion Matrix
7. Misclassified Images (Optional)

Students should write a brief inference (2–3 lines) for each figure.

18. Results

18.1 Performance Metrics (VGG16 – Primary Model)

Metric	Value
Training Accuracy	88.49%
Validation Accuracy(before fine-tuning)	85.82%
Validation Accuracy (after fine-tuning)	91.14%
Test Accuracy	91.05%
Precision	91.61%
Recall	91.05%
F1-score	91.13%
Training Time	44.86 min
Fine-Tuning Time	25.81 min
Total Parameters	14,781,642

18.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Precision (%)	Recall (%)	Training Time (min)
LeNet-5	83,126	52.38	52.71	52.38	0.80
AlexNet	36,926,730	10.00	1.00	10.00	5.35
VGG16	14,781,642	**91.05**	91.61	91.05	44.86
ResNet50	23,851,274	90.84	91.00	90.84	21.59

Summary: VGG16 achieves the best test accuracy (91.05%), closely followed by ResNet50 (90
LeNet-5 reaches only 52.38% because its shallow architecture was designed for small greyscale im
not CIFAR-10 RGB. AlexNet stagnates at 10% (random-chance baseline) due to architectural incom
patibility with upscaled CIFAR-10.

19. Plots and Inferences

19.1 Plot 1: CIFAR-10 Class Distribution

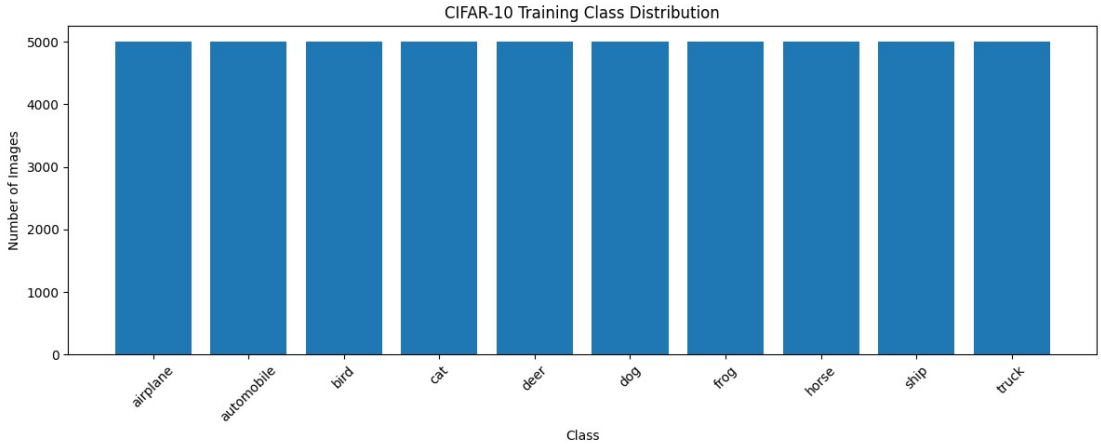

Figure 3:CIFAR-10 Training Set – Class Distribution

Inference: The CIFAR-10 training set is perfectly balanced, with each of the 10 classes containing exactly 5,000 samples. This balance ensures that the model is not biased towards any particular class and accuracy can be used directly as the primary evaluation metric without correction for cla imbalance.

19.2 Plot 2: Sample CIFAR-10 Images

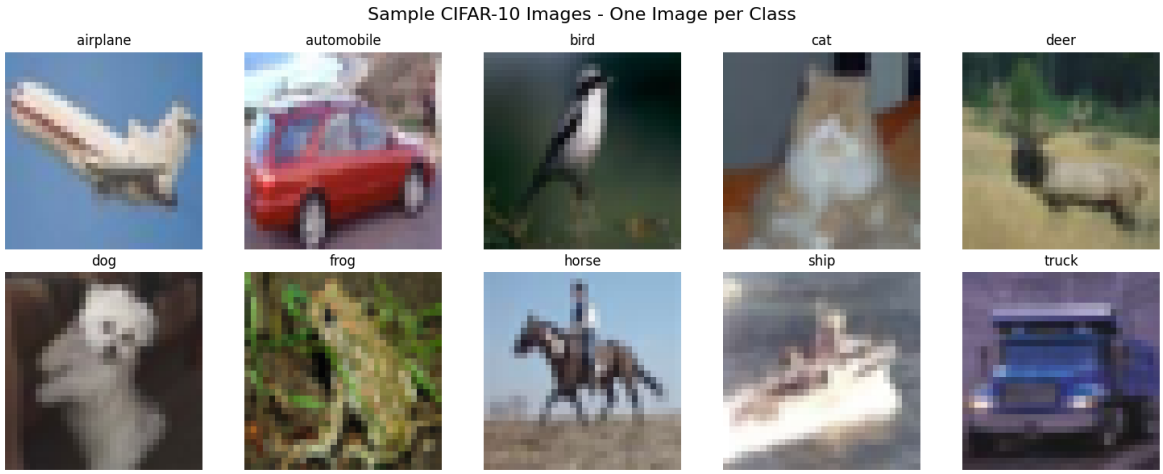

Figure 4: One Sample Image from Each CIFAR-10 Class

Inference: The sample images illustrate the diversity and challenge of the CIFAR-10 dataset. The low resolution (32 × 32 pixels) and high inter-class similarity (e.g., cat vs. dog, automobile vs. truck) make this a non-trivial classification task. Transfer learning from ImageNet-pretrained weights provides a significant advantage by leveraging general, generalised feature representations learned from millions of high-resolution images.

19.3 Plot 3: Training and Validation Accuracy (VGG16)

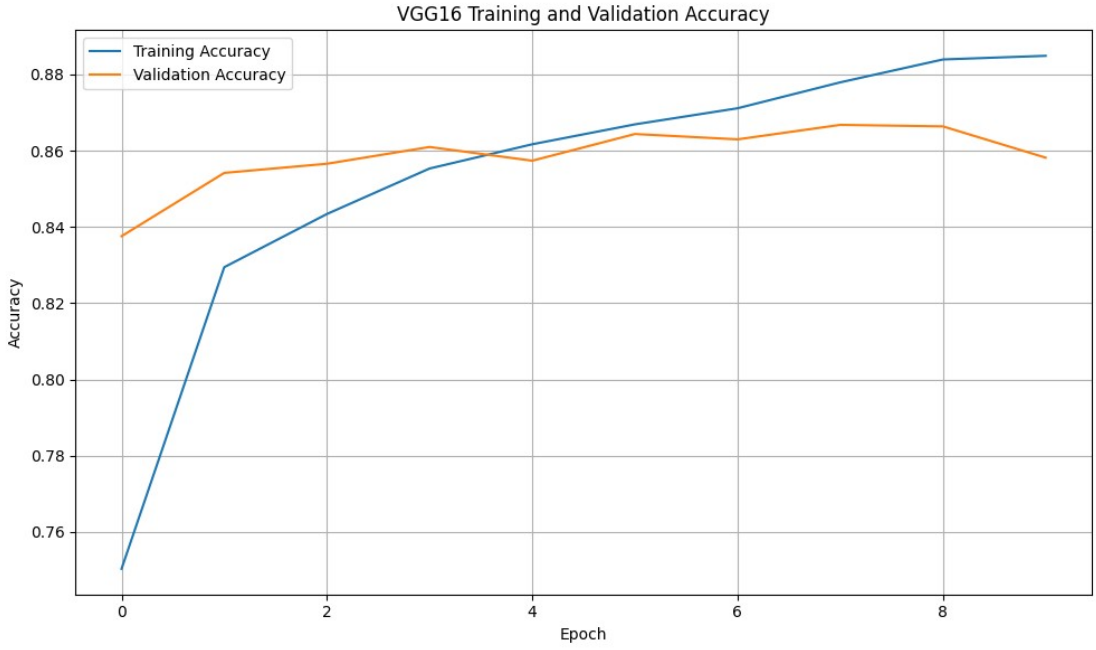

Figure 5: VGG16 Training and Validation Accuracy versus Epoch

Inference: Training accuracy rises steadily from approximately 75% to 88.5% over 10 epochs, while validation accuracy closely follows, reaching 85.8%. The small train–val gap (≈2.7%) indicates good generalisation without significant overfitting, which is expected since the convolutional base is frozen and only the newly added classification head is being trained.

19.4 Plot 4: Training and Validation Loss (VGG16)

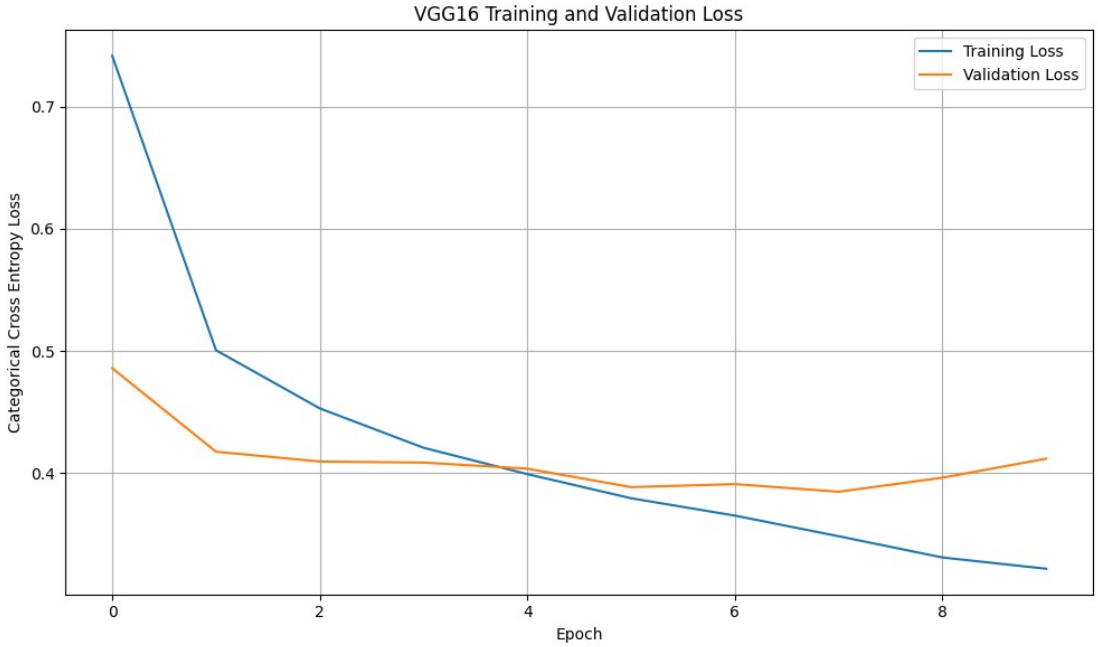

Figure 6:VGG16 Training and Validation Loss versus Epoch

Inference: Both training and validation losses decrease smoothly across 10 epochs, confirming stable optimisation with Adam (lr = 0.001). The two loss curves remain closely aligned, showing no divergence that would indicate overfitting. The monotone decrease also confirms that the learning rate is well chosen for fine-tuning the classification head.

19.5 Plot 5: Before vs After Fine-Tuning (VGG16)

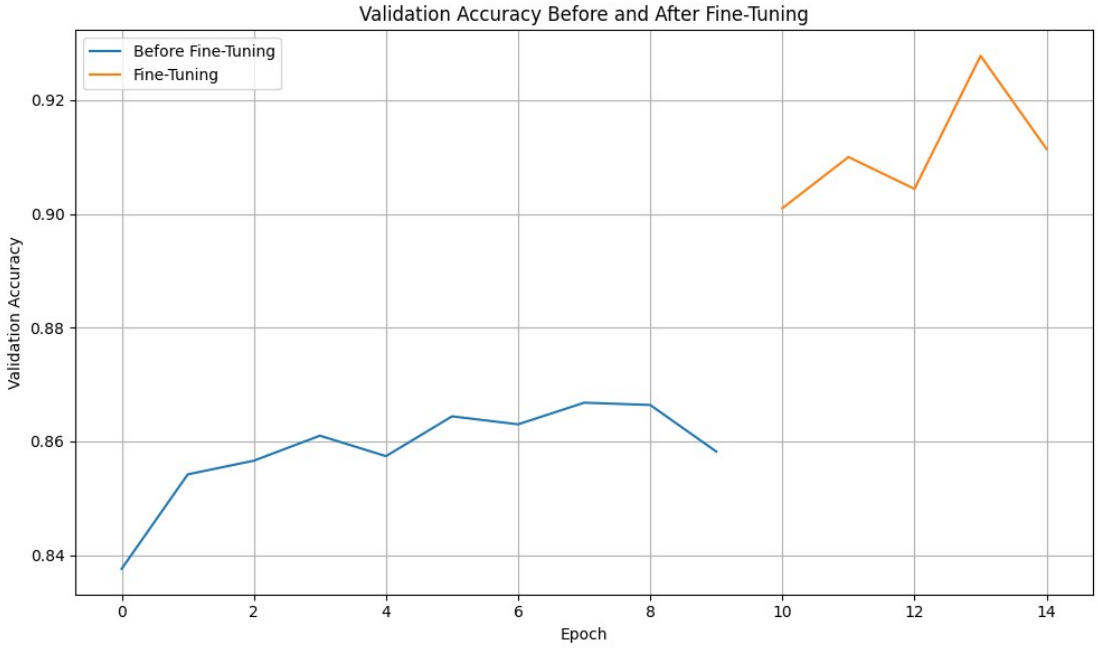

Figure 7:VGG16 Validation Accuracy: Feature Extraction vs. Fine-Tuning

Inference: Fine-tuning the last convolutional block improves the best validation accuracy from 85 to 91.14%, a gain of 5.3 percentage points. This demonstrates that higher-level VGG16 features, though

pretrained on ImageNet, can be meaningfully adapted to the CIFAR-10 domain. The gain justifies the additional 25.8 minutes of fine-tuning computation.

19.6 Plot 6: Confusion Matrix (VGG16)

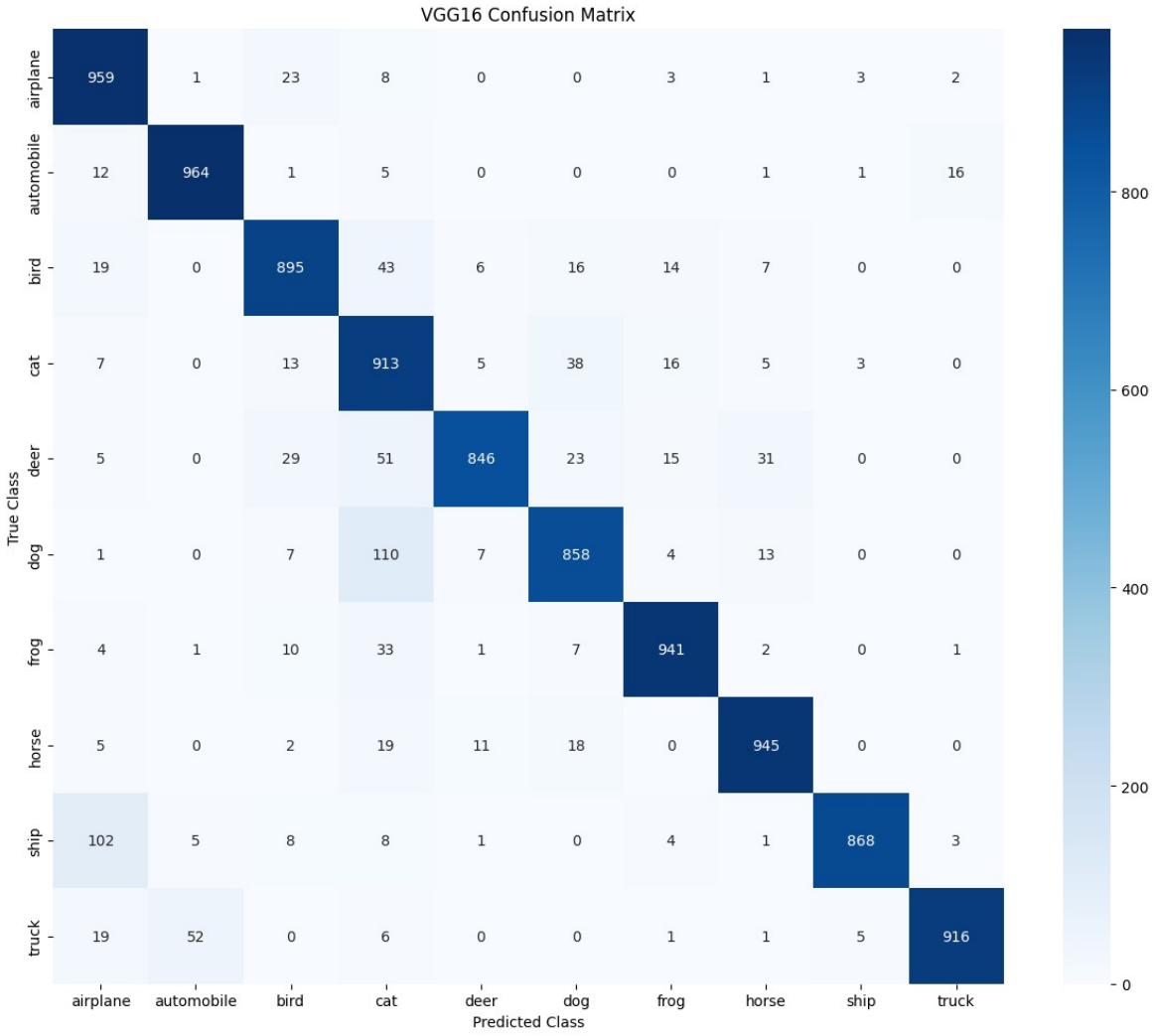

Figure 8:Confusion Matrix on the CIFAR-10 Test Set (VGG16 after Fine-Tuning)

Inference: The confusion matrix shows strong diagonal dominance, indicating consistently high per-class accuracy. The most frequent confusions occur between visually similar classes: cat ↔ dog and automobile ↔ truck. These errors are expected at 32 × 32 resolution. Classes such as automobile and ship achieve >96% accuracy owing to their distinctive visual features.

19.7 Plot 7: Per-Class Test Accuracy (VGG16)

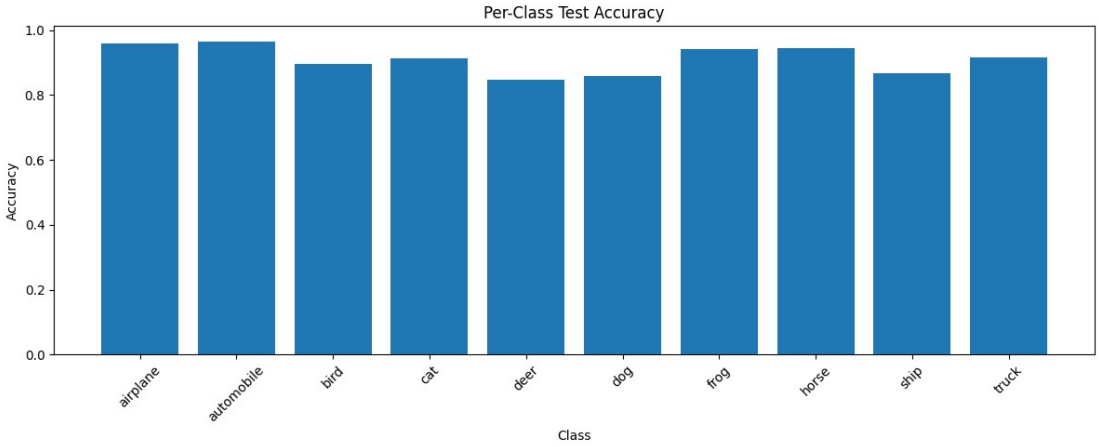

Figure 9: Per-Class Test Accuracy for VGG16 on CIFAR-10

Inference: Automobile (96.4%) and airplane (95.9%) achieve the highest per-class accuracy, while bird (84.6%) and dog (85.8%) are weakest. The lower performance on cat and deer stems from their visual overlap with dog and horse respectively. The overall accuracy of 91.05% reflects a well-performing model with no catastrophically weak class.

19.8 Plot 8: Misclassified Images (VGG16)

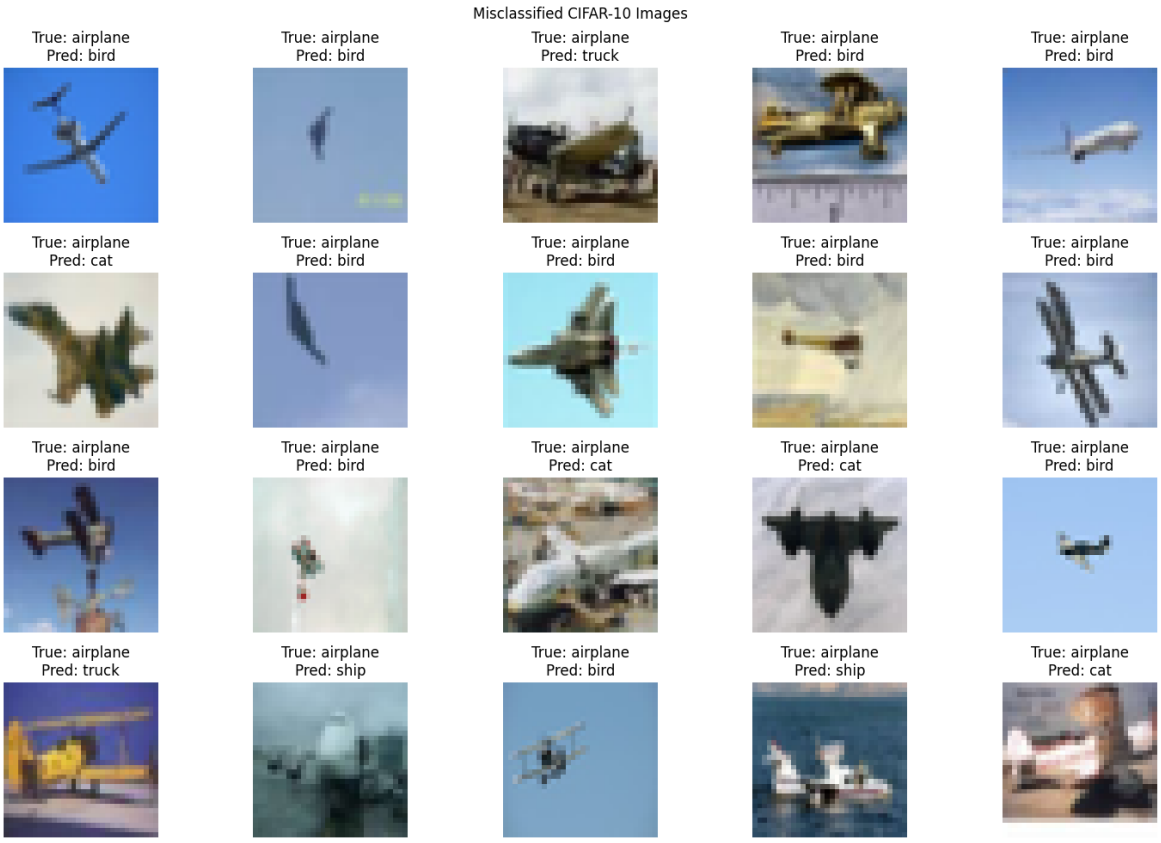

Figure 10: Representative Misclassified Images from the Test Set

Inference: Of the 10,000 test images, 895 (8.95%) are misclassified. The errors predominantly arise from challenging viewpoints, partial occlusion, and high inter-class similarity. Many misclassified sam-

ples are genuinely ambiguous even to a human observer, suggesting that the remaining error is cl
the irreducible Bayes error for CIFAR-10 at 32 × 32 resolution.

19.9 Plot 9: Architecture Accuracy Comparison

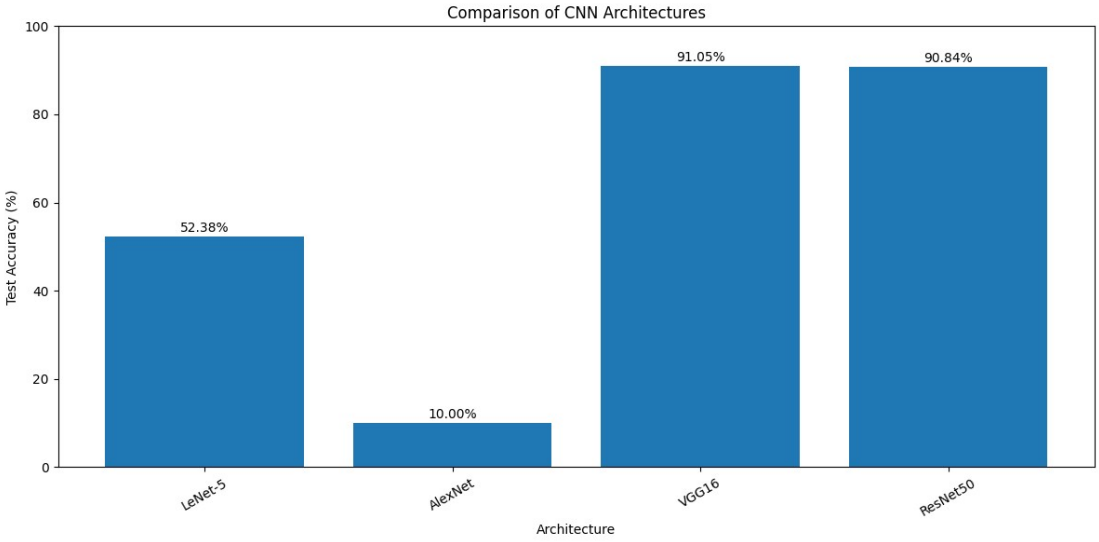

Figure 11: Test Accuracy Comparison Across CNN Architectures

Inference: VGG16 achieves the best test accuracy (91.05%), closely followed by ResNet50 (90.84%
LeNet-5 reaches only 52.38% – expected since its shallow 7-layer architecture cannot capture the
plex features in CIFAR-10. AlexNet stagnates at 10% (random chance) because its architecture is
ill-suited for CIFAR-10 without significant input adaptation.

19.10 Plot 10: Training Time Comparison

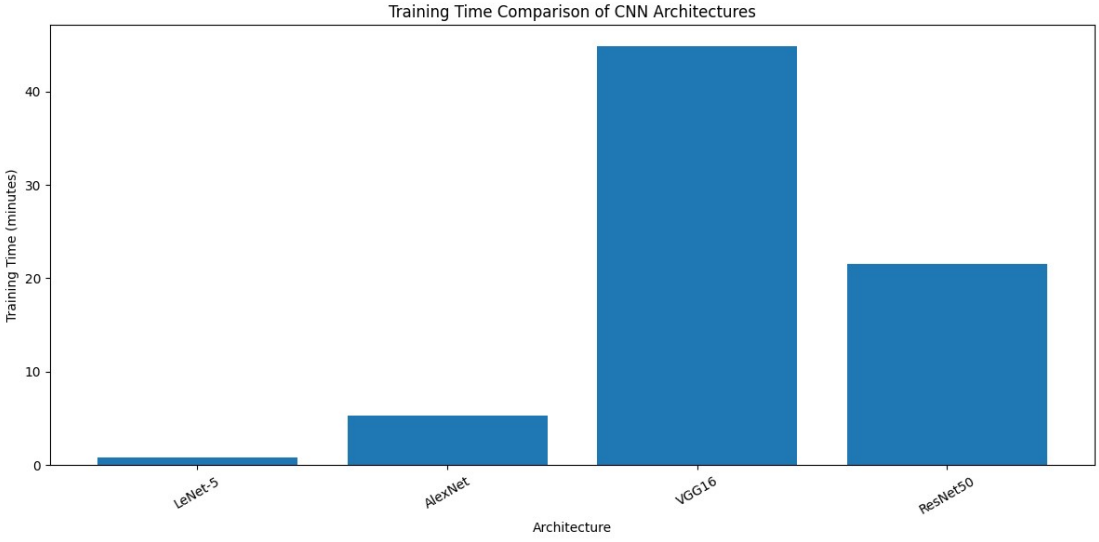

Figure 12: Training Time (minutes) Comparison Across CNN Architectures

Inference: VGG16 requires the longest training time (44.86 min) owing to its deep architecture, ev
with the convolutional base frozen. ResNet50 trains in 21.59 min despite having 50 layers, because
residual connections enable faster gradient propagation. LeNet-5 is the fastest (0.80 min) due to its
minimal depth and parameter count, illustrating the classic accuracy–efficiency trade-off.

19.11 Plot 11: Parameter Count Comparison

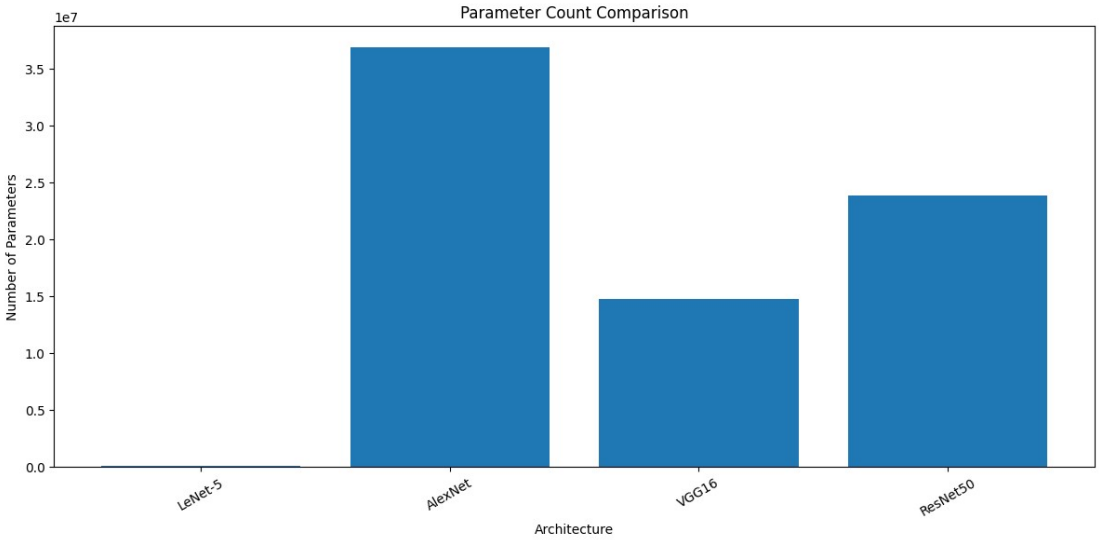

Figure 13: Number of Parameters Comparison Across CNN Architectures

Inference: AlexNet has the most parameters (36.9M) yet delivers the worst accuracy, confirming that parameter count alone does not determine performance. ResNet50 achieves near-VGG16 accuracy with 23.8M parameters, demonstrating the efficiency of residual learning. LeNet-5's 83K parameters are sufficient only within its intended domain.

19.12 Plot 12: Combined Architecture Comparison

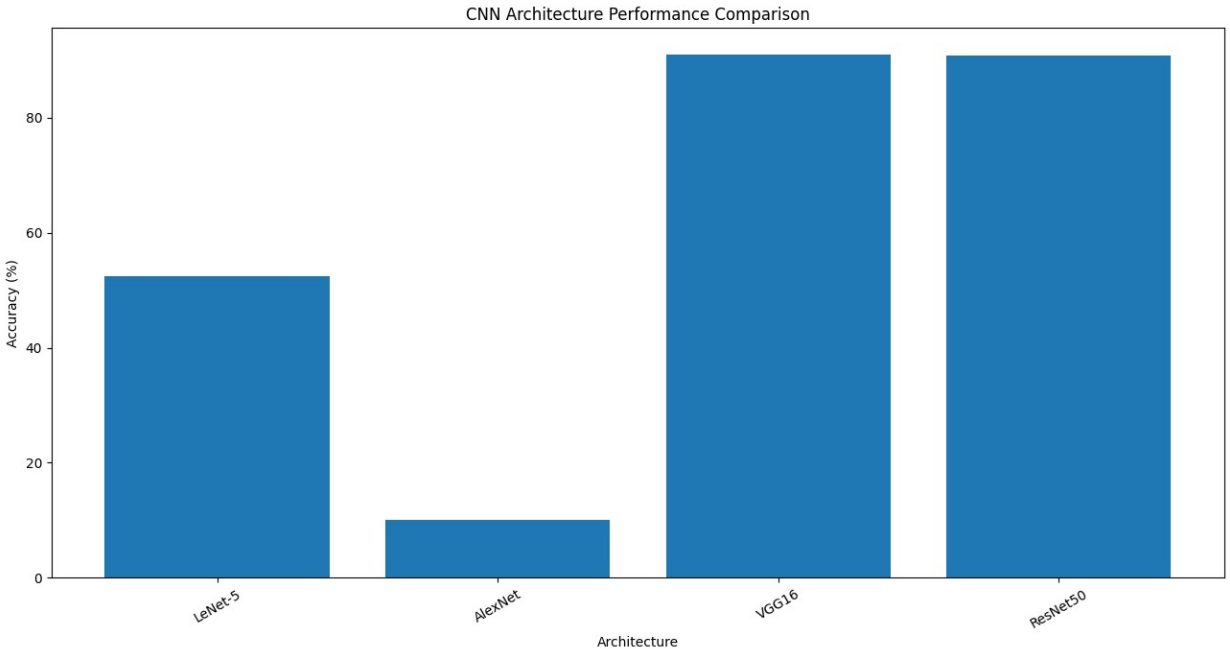

Figure 14: Combined Comparison: Accuracy, Parameters and Training Time

Inference: This combined chart reveals that VGG16 and ResNet50 occupy a similar accuracy tier (≈91%), but ResNet50 achieves this with fewer parameters and shorter training time, making it more efficient in resource-constrained environments. AlexNet is the worst performer across all three dimensions simultaneously, confirming it is unsuitable for CIFAR-10 without significant architectural adaptation.

14. Kaggle Notebook

The complete implementation of this experiment is available in the following Kaggle notebook:

kaggle notebook link

20. Discussion Questions

Answer the following questions.

1. **Why is AlexNet considered a breakthrough in deep learning?**
   AlexNet was considered a breakthrough because it showed that deep CNNs could achieve ver
   high performance on large-scale image classification problems.
   One important reason was the use of GPUs, which made it practical to train a much deeper
   network on a large image dataset. It also used ReLU activation, which helped the network train
   faster compared with traditional activations like sigmoid.
   AlexNet also used techniques such as Dropout and data augmentation, which helped reduce
   overfitting. Its success showed that increasing the depth of neural networks along with proper
   training techniques could give a major improvement in image recognition.
2. **Why does VGG16 use only** 3 × 3 **convolution filters?**
   VGG16 mainly uses small 3 × 3 filters because several small filters can achieve a similar rece
   field as a larger filter while introducing more nonlinearities between the layers.
   For example, two 3 × 3 convolutions can cover a similar area to a 5 × 5 convolution, but they
   fewer parameters.
   Also, using multiple 3 × 3 layers allows the network to learn more complex features step by s
   So, instead of using one large filter, VGG16 uses multiple small filters to make the network de
   and more efficient.
3. **Explain the advantages of the Inception module.**
   The main idea behind the Inception module is that we do not know beforehand whether a use
   feature will be large, small, wide, or narrow.
   So, instead of choosing only one filter size, the Inception module performs different operation
   parallel, such as:
   - 1 × 1 convolution
   - 3 × 3 convolution
   - 5 × 5 convolution
   - Pooling

   Their outputs are then combined.
   This allows the network to capture features at different scales. The 1 × 1 convolutions are also
   useful for reducing the number of channels before expensive convolutions, which helps control
   computation.
4. **What is the purpose of residual learning?**
   Residual learning was introduced to make it easier to train very deep networks.
   As networks become deeper, simply adding more layers can actually make training difficult. The
   problem is not only overfitting; the network may also have difficulty learning a good mapping
   because of issues such as vanishing gradients and optimization difficulty.
   ResNet solves this using a shortcut connection. Instead of forcing the layers to learn the complete
   mapping, they learn the difference, or residual, between the input and the desired output.
   The basic idea is:

   $$\text{Output} = F(x) + x$$

   Here, x is passed directly through the shortcut connection.
   This makes it easier for information and gradients to flow through very deep networks.
5. **Differentiate LeNet and ResNet.**
   LeNet is an early and relatively simple CNN architecture, mainly designed for handwritten dig
   recognition.

ResNet is a much deeper CNN architecture designed to solve the difficulty of training very dee
networks.
The main differences are:

LeNet	**ResNet**
Relatively shallow	Can be very deep
Designed for simple image recog-nition	Designed for complex image tasks
Uses basic convolution and pool-ing layers	Uses residual/shortcut connec-tions
No residual blocks	Uses residual blocks
Suitable for small images and simpler tasks	Suitable for large-scale image recognition

The important difference is that ResNet uses shortcut connections, which allow the network to
become much deeper without making training unnecessarily difficult.

6. **What is Transfer Learning?**
   Transfer learning means using the knowledge learned by a model on one dataset or task for a
   related task.
   For example, if MobileNetV2 or ResNet has already been trained on ImageNet, it has learned
   useful features such as edges, textures, shapes and object patterns.
   Instead of starting from random weights, we can take this pretrained model and adapt it to ou
   own dataset.
   This is especially useful when our dataset is small because we do not need to learn all the bas
   visual features from scratch.
7. **Why is fine-tuning required?**
   A pretrained model has learned general features from its original dataset, but those features
   not be perfectly suited to our new dataset.
   Fine-tuning allows us to slightly adjust some of the pretrained layers so that they learn featur
   specific to our problem.
   For example, a pretrained model may know how to detect general edges and shapes, but our
   dataset may contain medical images or different types of objects. Fine-tuning helps the model
   adapt its learned features to those images.
   We normally use a small learning rate during fine-tuning so that we do not destroy the useful
   features that the model has already learned.
8. **Explain the difference between dilated convolution and transpose convolution.**
   They are used for different purposes.
   Dilated convolution increases the receptive field without increasing the number of filter param
   significantly. It introduces gaps between the filter elements, allowing the network to see a large
   area of the input.
   For example, a dilation rate of 2 spreads the filter over a larger region.
   Transpose convolution, on the other hand, is commonly used to increase the spatial size of fe
   maps. It is therefore useful in tasks such as image segmentation and image generation.
   So, simply:

   Dilated Convolution → Larger receptive field

   Transpose Convolution → Upsampling

9. **Why do pretrained models converge faster?**
   A pretrained model does not start with completely random weights. It already contains useful
   knowledge learned from a large dataset.
   For example, early layers may already know how to detect edges, corners and textures, while
   deeper layers may recognize more complex patterns.

Therefore, when we train it on a new but related dataset, the model only needs to adjust the existing knowledge instead of learning everything from the beginning.
Because it starts from a much better position, it usually reaches a good solution in fewer epoc than a model trained from scratch.

10. **Compare the computational complexity of LeNet and ResNet.**
LeNet has a much simpler architecture with fewer layers, fewer parameters and fewer computations. Therefore, it can be trained and executed with relatively low computational resources.
ResNet is much deeper and contains many more convolutional operations and parameters, so computational requirement is much higher.
However, ResNet is designed for much more complex image-recognition tasks and uses residua connections to make deep training possible.
So, the basic comparison is:

LeNet → Low computation, simple architecture

ResNet → Higher computation, deep architecture

The important point is that higher computational complexity in ResNet is intentional because allows the model to learn much more complex visual representations.

21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applie to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation. https://www.tensorflow.org
8. Keras Documentation. https://keras.io